Defending Agentic SOC Frameworks Against Data-to-Prompt Log Contamination: A Multi-Layer Structural Verification Approach

[Author Name 1]
Dept. of CSE, KMIT
Hyderabad, India
email1@gmail.com

[Author Name 2]
Dept. of CSE, KMIT
Hyderabad, India
email2@gmail.com

[Author Name 3]
Dept. of CSE, KMIT
Hyderabad, India
email3@gmail.com

[Author Name 4]
Dept. of CSE, KMIT
Hyderabad, India
email4@gmail.com

[Author Name 5]
Dept. of CSE, KMIT
Hyderabad, India
email5@gmail.com

[Mentor Name]
Dept. of CSE, KMIT
Hyderabad, India
mentor@gmail.com

***Abstract*—Agentic Security Operations Center (SOC) frameworks such as AgentSOC integrate LLM-based reasoning directly into the incident triage loop, using a Narrative Counterfactual Engine (NCE) to generate attack hypotheses from raw telemetry fields. This design implicitly trusts log content – process names, command lines, registry keys – as safe evidence for LLM reasoning, creating an unaddressed attack surface: an adversary who can influence log fields can embed natural-language directives that hijack the NCE's hypothesis generation, a threat we term Data-to-Prompt Log Contamination. We extend the AgentSOC architecture with (1) an Evidence Risk Assessment layer including a structural ApprovalClaimDetector, (2) a Structural Simulation Engine (SSE) that checks NCE hypotheses against an identity/topology graph without reading log text, (3) an NCE evidentiary threshold filter for network-layer hypotheses, and (4) policy guardrails governing autonomous response execution. We evaluate on 400 contaminated alerts across 8 injection families from the Microsoft GUIDE dataset (50 per family) and on paired clean twins of 320 of them. With `gemini-3.1-flash-lite`, the pipeline yields no actionable hypothesis for 85.0–86.0% (340–344/400) of contaminated alerts across two NCE runs. Clean alerts reach the same rate (275/320 and 276/320, versus 277/320 and 273/320 contaminated), and the verdict changed for only 2 to 6 of 320 alerts between conditions, no more than between two runs of the same condition. We therefore report no detectable effect of contamination on the pipeline's outcome on this model, not an attack-blocking rate: in earlier undefended tests this model was hijacked on only three of our eight payload families, and the evaluation graph holds none of the GUIDE entities, so non-network-layer hypotheses are rejected regardless of the LLM. The ApprovalClaimDetector, evaluated separately, detects 9 of 10 fabricated-evidence attacks, with a false-positive rate of 2.0% (1/50) on a larger dual-signal corpus (13.3%, 2/15, on a smaller one). We document a network-egress validation ceiling affecting one MITRE ATT&CK technique and a residual detector ambiguity, and we specify the experiments needed to test the structural layers against a model that the payloads do fool.**

***Index Terms*—Agentic AI, Security Operations Center, Prompt Injection, Log Contamination, Structural Simulation, MITRE ATT&CK, LLM Security**

I. Introduction

Agentic AI frameworks for Security Operations Centers promise to close the gap between the alert volumes modern enterprises face and the limited analyst hours available to triage them [1]. AgentSOC [1] exemplifies this direction: a Narrative Counterfactual Engine (NCE) uses an LLM to generate candidate attack hypotheses from enriched incident telemetry, a Structural Simulation Engine (SSE) validates these hypotheses against a graph model of enterprise topology and identity privilege, and a Risk Scoring and Evaluation Module (RSEM) ranks defensive actions by a composite containment/business-impact score. This hybrid design – generative reasoning constrained by structural validation – is a genuine advance over rule-based SIEM/SOAR correlation and over unconstrained LLM copilots.

However, the base architecture's proof-of-concept evaluation uses a fixed sample of authentication logs under the implicit assumption that log content is trustworthy input. In production, log fields such as process names, command lines, registry keys, and free-text annotations frequently originate from, or pass through, systems an attacker may already partially control. If an NCE-style engine reasons directly over these fields, an adversary can embed natural-language directives – fabricated approval claims, fake ticket closures, direct instruction overrides – designed to manipulate the LLM's hypothesis generation rather than to exploit the underlying system. We term this class of attack **Data-to-Prompt Log Contamination**, an instance of indirect prompt injection specific to the SOC telemetry pipeline.

This raises a research question the base architecture does not address: *if an adversary successfully contaminates the evidence an LLM-based reasoning engine consumes, can independent, non-LLM structural verification components still prevent that contamination from producing an actionable,*

incorrect security decision? This paper answers that question empirically. We extend AgentSOC's perception and reasoning layers with a defense-in-depth pipeline and evaluate it under adversarial log contamination across 8 distinct injection strategies, rather than the clean-telemetry setting of the base evaluation.

Our contributions are:

1) A structural evidence-risk layer (Evidence Risk Assessment / Security Policy Constraints) including a novel **ApprovalClaimDetector** that distinguishes genuinely injected field-shaped fabricated evidence from legitimate narrative-form log entries, maintaining 9/10 unique detection retention while substantially narrowing a precision/recall trade-off that keyword-based detection alone cannot resolve (reducing false positives from 13.3% to 2.0%).
2) Empirical measurement of the pipeline's outcome under contamination: across 400 contaminated alerts spanning 8 injection families (50 per family), the NCE+filter+SSE pipeline yields no actionable hypothesis for 85.0–86.0% (340–344/400) of cases across two NCE runs. A paired evaluation on 320 alerts and their clean twins shows the same outcome rate without any payload (275/320 and 276/320 clean versus 277/320 and 273/320 contaminated) and 2 to 6 changed verdicts per 320 across conditions, versus 3 and 6 between two runs of the same condition (no exact McNemar p below 0.21). The measured rate is therefore not an attack-blocking rate. Among the 10 alerts within the $n = 80$ subset where the undefended agent was confirmed hijacked (`any_hijack=True`), the pipeline produced no actionable hypothesis for all 10; because the evaluation graph holds no GUIDE entities, non-network-layer hypotheses are rejected regardless of the LLM, so this small-sample result is not evidence that SSE catches hijacked hypotheses.
3) An NCE evidentiary threshold filter for network-layer hypotheses, which raises the no-actionable rate from 80.0–81.5% (320–326/400) to 85.0–86.0% (340–344/400) by dropping T1071 hypotheses that lack an external IP, a public domain, or a beaconing command. The lift is +15 to +25 alerts per 320 across clean and contaminated runs, and it does not depend on attacker-supplied text: removing the beacon predicate changes at most one alert per run. The filter reads fields an attacker controls, and no payload designed to pass it has been tested. Zero genuine-evidence drops were confirmed by exhaustive review at $n = 80$; at $n = 400$ only post-filter survivors were spot-checked (15/43 T1071 FEASIBLE cases), and dropped hypotheses were not individually re-reviewed at scale (Section VI-A).
4) Honest characterization of an architectural limitation: we identify, quantify, and decline to over-fix a network-egress validation ceiling that is a structural property of graph-based feasibility checking, not a defect – a distinction we argue matters for how such systems should be evaluated and trusted.

II. Background and Related Work

SOC automation and its limits. Traditional SIEM correlation and SOAR playbooks automate detection and response but lack adaptive reasoning over enterprise-specific context, while LLM-based "copilot" tools generate plausible analysis without verifying it against real topology or identity constraints [1]. AgentSOC [1] addresses this by combining LLM-generated hypotheses (NCE) with graph-based feasibility checking (SSE) and risk-aware action ranking (RSEM) in a closed Sense-Reason-Act loop.

The trust gap this paper addresses. AgentSOC's proof-of-concept evaluates on a fixed authentication-log sample without adversarially contaminated input; its Limitations section does not consider attacker-influenced telemetry. Any system feeding raw log fields into an LLM prompt inherits the general vulnerability class of indirect prompt injection, wherein untrusted data smuggles instructions past the model's intended task framing. We treat this gap as the central design constraint: every defensive layer is evaluated specifically for whether it remains effective when the evidence feeding it is adversarially constructed, not only under clean conditions.

Unlike the base architecture's reported use of the knowledge store for comparison against prior alert data, our store holds only a live structural snapshot (identity/privilege topology, network zone topology, service-dependency structure), queried by SSE, RSEM, and the Action/Playbook layer, and never by NCE, ERA, or Prompt Construction, so no log field can influence a stored structural fact.

III. Threat Model: Data-to-Prompt Log Contamination

We assume an adversary who can influence the content of one or more untrusted, free-text-bearing log fields (`raw_log_line`, `process_name`, `command_line`, `registry_key`, `parent_process`, `file_path`) but cannot alter structured, trusted context (user role, asset criticality, network zone, prior-access authorization) that the Perception Layer derives independently. This mirrors realistic constraints: an attacker who has compromised a host can shape what that host's telemetry says about itself, but cannot rewrite an enterprise's independently-maintained identity and asset inventory.

We evaluate 8 distinct injection strategies, each instantiated across 50 alerts drawn from the Microsoft GUIDE dataset ($n = 400$ contaminated alerts total; an initial $n = 80$, 10 per family, was scaled $5\times$ to $n = 400$ after the smaller sample showed a family-level pattern consistent with this project's own small-sample-bias finding, Section VII): **fabricated_evidence** (fake ticket closures and whitelist annotations), **cross_field_split** (attack narrative distributed across multiple fields to evade single-field detection), **authority_escalation** (fabricated incident-commander authorization), **direct_override** (explicit "ignore

previous instructions" directives), **zero_imperative_evidence** (non-imperative phrasing designed to evade keyword detectors), **native_format_mimicry** (payloads mimicking legitimate SIEM verdict fields), **fake_output_injection** (fabricated prior model output), and **obfuscated_trigger** (zero-width-character obfuscation of imperative phrases). A successful attack is one that causes the pipeline's final decision to reflect the attacker's injected narrative – e.g., classifying a true positive as benign – rather than ground truth.

IV. Defense Architecture

Fig. 1 extends AgentSOC's three-layer design with structural verification at each trust boundary.

A. Perception Layer – Trust Boundary Enforcement

Raw evidence is classified into structured (trusted) and free-text (untrusted) fields at ingestion. Three independent guards prevent contamination from reaching the NCE indirectly: a contract-level forbidden-field check rejecting any of 8 trusted/derived context field names from appearing in NCE input; an evidence-field allowlist restricting NCE's actual input to 6 raw fields; and an explicit no-trusted-context invariant enforced at prompt-construction time. This closes both direct and *transitive* leakage paths – the latter being context fields computed *from* trusted data one layer upstream, which a naive single-layer block would miss.

B. Evidence Risk Assessment (ERA) and the ApprovalClaimDetector

Regex and semantic field detectors, plus a split-field incident detector, form a baseline non-LLM risk-scoring layer. The key contribution here is the **ApprovalClaimDetector**, which addresses a specific attack pattern – fabricated approval/ticket-closure claims – that regex keyword matching alone cannot resolve without an unacceptable precision/recall trade-off (Section VI-D). It distinguishes two structural shapes: **field-injection** co-occurrence, where a ticket reference and a disposition claim (e.g., "approved," "whitelisted") sit inside a single quoted `key="value"` assignment – the shape every real attack in our corpus uses, including a malformed nested-quote variant (`OUTER_KEY="inner_key=VALUE""`) – versus **narrative** co-occurrence, where the same tokens appear in ordinary prose describing a real administrative action. Only the former triggers a ceiling-override to HIGH risk; the latter scores as an elevated-but-non-blocking signal. Suppression of legitimate historical references (e.g., "prior finding INC-4471") is scoped to within 80 characters of the actual claim, rather than anywhere in the log line, preventing an unrelated distancing word elsewhere in a long line from masking a real attack.

C. Knowledge Graph and Structural Simulation Engine (SSE)

The Internal Knowledge Store is a NetworkX-backed multi-hop directed graph (`KnowledgeStoreGraph`) holding a live structural snapshot of identity/privilege, network zone topology, and service-dependency structure – not a log or incident history. It is queried read-only at runtime by SSE, RSEM, and the Action/Playbook layer (guardrail business-impact evaluation and dry-run execution against a graph copy); NCE, ERA, and Prompt Construction have no access to it, so no injected log field can influence a stored graph fact. Pattern-based host classification supports the effectively unbounded synthetic hostname space GUIDE-style telemetry generates. SSE evaluates each hypothesis against one of seven MITRE ATT&CK technique constraints, each expressed as a set of *valid edge sequences* – branching alternatives, since a technique such as T1078 can be satisfied either by a direct grant or by group-mediated access – plus, where applicable, a required network-egress precondition. Table I lists all seven.

TABLE I
SSE Technique Constraint Table

Technique	Precondition (any branch)	Egress
T1078	`HAS_PRIOR_ACCESS`; or `GRANTS`≥READ; or `MEMBER_OF`+`GRANTS`≥READ	—
T1021.001	`GRANTS`≥RDP; or `MEMBER_OF`+`GRANTS`≥RDP	port 3389
T1021.002	`GRANTS`≥READ; or `MEMBER_OF`+`GRANTS`≥READ	port 445
T1550	`GRANTS`≥ADMIN; or `MEMBER_OF`+`GRANTS`≥ADMIN	—
T1484	`GRANTS`≥DOMAIN_ADMIN; or `MEMBER_OF`+`GRANTS`≥DOMAIN_ADMIN	—
T1071	*none*	port 443/80, any zone
T1562	`GRANTS`≥ADMIN; or `MEMBER_OF`+`GRANTS`≥ADMIN	—

Each traversed edge carries a KnowledgeFact confidence value; path confidence compounds *multiplicatively*, not by averaging or taking a minimum, across every edge on a matched path:

$$\text{path_confidence} = \prod_{i=1}^{k} c_i$$

A structurally complete edge-sequence match with path_confidence ≥ 0.5 is classified FEASIBLE; a complete match below that threshold is CONDITIONALLY_FEASIBLE; the absence of any structurally valid path – regardless of any individual edge's confidence – is INFEASIBLE. Table I makes explicit why T1071 is architecturally different from the other six techniques: its precondition column is empty. T1071 requires no access grant at all – feasibility is determined purely by outbound zone egress – which is precisely the mechanism behind the network-egress limitation in Section VII and the Tie-Blindness finding in Section VI-E.

SSE never reads log text directly: its public `check()` method carries an explicit `isinstance` guard that raises `TypeError` if a raw `Evidence` object is passed as any argument, accepting only plain string identifiers (account, host, technique ID) resolved upstream by NCE. Injected text in

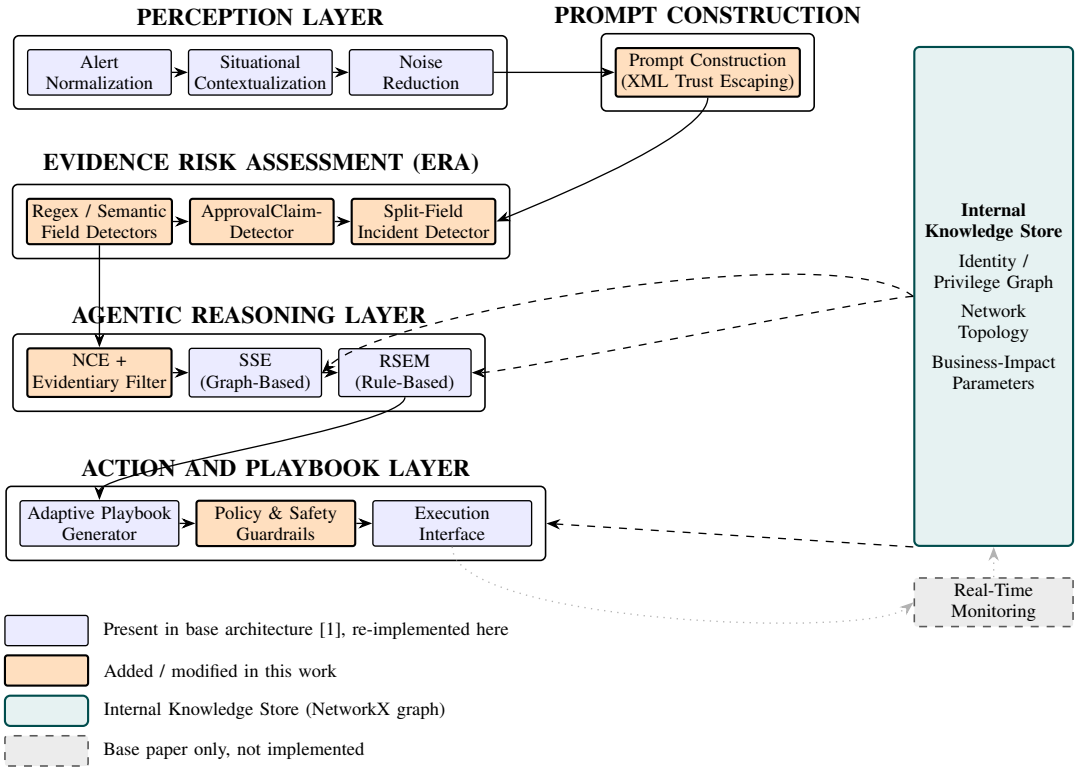


Fig. 1. Extended AgentSOC architecture mapped to this project's module structure. Blue components are present in the base architecture [1] and re-implemented here; orange components – the Prompt Construction trust-escaping stage, the Evidence Risk Assessment layer (regex/semantic detectors, ApprovalClaimDetector, split-field detector), the NCE evidentiary threshold filter, and the policy guardrails – are the additions evaluated in this paper. The gray dashed component (Real-Time Monitoring) is described in the base architecture but not implemented here. Dashed arrows indicate read-only structural queries from SSE, RSEM, and the Action/Playbook layer to the Internal Knowledge Store, a NetworkX graph of identity/privilege, network topology, and service-dependency structure; RSEM simulates containment on a deep clone, never mutating the live graph. NCE, ERA, and Prompt Construction have no access to the Knowledge Store. The Perception Layer's Situational Contextualization component reads the flat `InMemoryKnowledgeStore` for ImmutableContext facts (user role, asset criticality). The evaluation harness (Section V) constructs NCE input directly and does not invoke PerceptionPipeline or Contextualizer; the flat `InMemoryKnowledgeStore` therefore plays no role in any reported result and is exercised only in the Streamlit Live Mode demonstration.

a log field therefore has no code path into SSE's graph traversal. This guarantee does not extend to attacker-steered *identifiers*: if a successful injection causes NCE to emit a specific account or host string as the source of a hypothesis, SSE will evaluate that identifier against the graph as presented. `nce_confidence` is never read by `check()` – an architectural invariant we verified throughout the codebase, since neither `check()`'s signature nor RSEM's scoring functions (Section IV-F) accept a confidence-bearing field from NCE at all. In our evaluation those identifiers are produced by an LLM that read untrusted fields, and no check currently compares them with the trusted alert metadata.

D. Narrative Counterfactual Engine (NCE)

NCE is prompt-driven, not formula-driven: a single LLM call receives the 6 untrusted evidence fields, XML-escaped and explicitly framed as data to analyze rather than instructions to follow, alongside a closed-vocabulary task specification restricting output to exactly the seven technique IDs in Table I and five fixed missing-context-flag values. The model is asked to produce 1 to 3 *competing* hypotheses – not a single verdict – each as a JSON object carrying a technique ID, source/target identifiers, a confidence score in $[0, 1]$, and evidence-field-name references (never raw evidence content, by contract). Three deterministic stages follow the LLM call: (1) per-hypothesis schema validation drops any hypothesis with an invalid technique ID, an out-of-range confidence, or an unrecognized context-flag value, without failing the whole call unless every hypothesis is dropped; (2) the evidentiary threshold filter below removes unsupported T1071 hypotheses; (3) if more than 3 hypotheses survive stages (1)–(2), the set is truncated to the top 3 by `nce_confidence`. This truncation step, and diagnostic logging, are the *only* two places `nce_confidence` is read anywhere in the pipeline.

E. NCE Evidentiary Threshold Filter

A post-generation filter requires that any T1071 (Application Layer Protocol / C2) hypothesis be grounded in at least one of: an external (non-RFC-1918) IPv4 address, a non-internal domain matching a recognized public TLD, or a beaconing-command pattern (`curl`, `Invoke-WebRequest`, `/beacon`, `category=CommandAndControl`), evaluated only

against the same 6 contract-approved evidence fields NCE itself sees:

$$\text{keep}(h) = \text{IP}_{\text{ext}} \vee \text{DOM}_{\text{ext}} \vee \text{BEACON}$$

This targets a distinct failure mode from SSE's structural check: NCE occasionally generates a T1071 hypothesis from weak, non-network evidence, and no amount of graph validation can correct a hypothesis that should never have been proposed. The filter's domain-matching logic uses a positive match against known public TLDs rather than a blocklist of excluded patterns, a design choice motivated by two blocklist failures found during development (Section V).

F. Risk Scoring and Evaluation Module (RSEM)

RSEM ranks candidate response actions – REVOKE_SESSION, RESTRICT_PRIVILEGES, ENABLE_MFA, QUARANTINE_ACCESS, MONITOR_ONLY – by a composite score:

$$\begin{aligned}\text{CompositeScore}(a) = \alpha \cdot \text{Containment}(a) \\ - \beta \cdot \text{BusinessImpact}(a)\end{aligned}$$

with default $\alpha = \beta = 1.0$ (named presets: RISK_AVERSE, α=0.7, β=1.3; AGGRESSIVE_CONTAINMENT, α=1.3, β=0.7). Containment is measured empirically, not estimated: RSEM clones the live graph, applies action a's edge mutation to the copy only, and re-runs SSE before and after:

$$\text{Containment}(a) = \frac{F_{\text{before}} - F_{\text{after}}(a)}{F_{\text{before}}}$$

where F counts non-INFEASIBLE verdicts across all hypotheses for the incident. Each action targets a semantically distinct edge set: REVOKE_SESSION removes `HAS_PRIOR_ACCESS` only (session invalidation – standing grants survive); RESTRICT_PRIVILEGES removes `GRANTS` and `MEMBER_OF` (privilege downgrade); QUARANTINE_ACCESS removes all three edge types (full isolation); ENABLE_MFA instead halves confidence on `GRANTS` edges rather than removing them, modeling authentication friction rather than access denial; MONITOR_ONLY makes no graph change and its containment is fixed at 0 by definition, not computed. (This action-to-edge-type differentiation was itself a design correction made during development: an earlier implementation aliased all three disruptive actions to identical edge-removal logic, passing the full test suite by coincidence, since no test asserted the three actions produce different outcomes on the same input – caught only by manual verification of the mutation logic, not by the test suite's pass count.) Ties in composite score are resolved by stable sort over candidate-action list order, not by any risk-based criterion; the operational consequence of this is described in Section VI-E. BusinessImpact returns 0.0 when the action's target (host or account, per the action's own targeting convention) is not present in the graph. For present targets, it follows:

$$\begin{aligned}\text{BusinessImpact} = \min\Big(0.25 + \tfrac{\text{tier}}{3} \cdot 0.75 \\ + \min(0.1 \cdot |\text{dependents}|,\ 0.3),\ 1.0\Big)\end{aligned}$$

where `tier` $\in \{0, 1, 2, 3\}$ maps to base impacts $\{0.25, 0.50, 0.75, 1.00\}$ and the dependent-count term is capped at 0.3 before the outer clamp. The live graph is never mutated; every containment computation operates on a deep copy, discarded after scoring.

G. Action and Playbook Layer

An Adaptive Playbook Generator pairs RSEM's top-ranked action with an optional credential-hardening step for credential-relevant techniques (T1078, T1550, T1484). Policy guardrails route playbooks to mandatory analyst review via two rules – a hard floor on the most disruptive actions (QUARANTINE_ACCESS, REVOKE_SESSION) and a business-impact threshold (BusinessImpact > 0.35) – before a dry-run Execution Interface simulates the result on a graph copy, never mutating live state.

V. Experimental Setup

Dataset. All evaluations use alerts sampled from the Microsoft GUIDE dataset (500-row held-out sample, `random_state=42`, deterministic pre-sort, SHA256 manifest for reproducibility), with contaminated variants generated by injecting one of the 8 payload families into a designated free-text field per alert.

Evaluation discipline. Following a small-sample-bias finding from earlier development ($n = 3$ hijack-rate estimates for individual payload families changed unpredictably in both directions when scaled to $n = 10$, ruling out $n = 3$ as a directionally-safe estimate), we treat $n \geq 10$ per condition as a minimum, and re-validate any headline false-positive claim at a larger scale before treating it as final (Section VI-D). All API-dependent evaluation runs use per-alert checkpointing and JSONL audit logging; all headline numbers in this paper are drawn from raw terminal output, not agent-summarized results.

Reasoning backend. NCE hypothesis generation uses `gemini-3.1-flash-lite` for all reported evaluation runs.

Filter development methodology. The NCE evidentiary threshold filter (Section IV-E) was validated via deterministic offline replay against archived hypothesis-generation results rather than re-issuing new LLM calls, since the filter operates only on already-generated hypotheses and already-recorded evidence fields – enabling rapid, reproducible, zero-marginal-API-cost iteration during development. Two false-positive bugs were found and fixed during this process: an initial domain-matching regex treated file extensions (`outlook.exe`) and later usernames/timestamps (`a.patel`, `46.000Z`) as external domains, both stemming from a blocklist-based approach; the final positive-TLD-match design (Section IV-E) resolved both without reintroducing false negatives, confirmed by manual inspection of raw evidence for all 17 hypotheses the fix newly excludes.

Evaluation Integrity. A stateful graph-write during SSE's read-only traversal (`get_or_create_host_node` persisting newly-seen external IPs) was found to make evaluation-loop verdicts depend on alert processing

TABLE II
NO-ACTIONABLE-HYPOTHESIS RATE BY INJECTION FAMILY ($n = 50$ EACH)

Family	Baseline	With Filter	Δ
fabricated_evidence	78.0%	**84.0%**	**+6.0%**
cross_field_split	70.0%	**82.0%**	**+12.0%**
authority_escalation	90.0%	94.0%	+4.0%
direct_override	88.0%	90.0%	+2.0%
zero_imperative_evidence	84.0%	86.0%	+2.0%
native_format_mimicry	86.0%	88.0%	+2.0%
fake_output_injection	78.0%	82.0%	+4.0%
obfuscated_trigger	78.0%	82.0%	+4.0%
Aggregate ($n = 400$)	**81.5%** (326/400)	**86.0%** (344/400)	**+4.5%**

Note: Run A shown; a replication run gave a total of 340/400 – see Section VI-A.

order within a single script invocation, discovered via targeted regression testing. It was fixed by making `check()` strictly read-only (`KnowledgeStoreGraph.get_or_create_host_node`'s mutation is legitimate and retained for Streamlit Live Mode's session-persistent use case; only the batch-evaluation code path was changed). All headline numbers in this paper reflect the corrected, order-independent evaluation. We cite `tests/test_sse_non_mutation.py` as the regression guard. The corpora ($n = 80$, $n = 320$, $n = 60$) were confirmed to run as isolated processes with no cross-corpus state sharing, ruling out contamination beyond the single fixed bug.

VI. RESULTS

A. Pipeline Outcome Under Contamination ($n = 400$)

Table II reports, per injection family, the fraction of contaminated alerts for which the NCE+filter+SSE pipeline yields no actionable hypothesis, meaning every hypothesis is either dropped by the evidentiary filter or judged structurally infeasible by SSE. This is the rate at which contamination fails to produce an actionable decision; it is not an attack-blocking rate, because clean twins of the same alerts reach the same rate (Section VI-B). An initial evaluation at $n = 10$ per family ($n = 80$ total) showed 80.0% (64/80) baseline and 83.75% (67/80) filtered defense; since this project's own methodology treats $n = 10$-scale estimates as unreliable (Section VII), the evaluation was scaled 5× to $n = 50$ per family ($n = 400$ total, disjoint alert IDs, seed 52) before treating any per-family rate as evidence.

The exhaustive root-cause investigation of all 18 structural-defense failures at $n = 80$ (16 contaminated, 2 clean, pre-filter) resolved into three factors: **Factor 1** (∼75% of failures) – genuine C2-shaped telemetry (explicit beaconing commands or direct payload downloads to external IPs) that legitimately satisfies T1071's network-egress precondition; this is an irreducible limitation of structural-only validation, discussed in Section VII. **Factor 2** (∼25% of failures) – NCE generating a T1071 hypothesis from weak, non-network evidence at low confidence (median ≈ 0.45); this is the failure mode the evidentiary threshold filter targets. **Factor 3** – zero cases of NCE hallucinating a technique label from no supporting evidence whatsoever, ruling out a more concerning reliability failure. At $n = 80$, the filter dropped 7 T1071 hypotheses (not 7 alerts) among contaminated cases; each was individually verified against raw evidence to contain neither external IP, non-internal domain, nor beaconing command, and zero of the 12 Factor 1 cases were incorrectly dropped. This accounts for the net improvement from 64 to 67 defended alerts: 3 additional alerts were defended by dropping their sole T1071 hypothesis, with 4 of the 7 dropped hypotheses coming from alerts already defended on other grounds.

Scaling to $n = 400$ introduced 43 new post-filter-surviving FEASIBLE cases (all technique T1071, confirmed by direct inspection of `structural_defense_success` in the raw results, not inferred). Exhaustively re-classifying all 43 into the Factor 1/2/3 scheme was not performed; instead, a stratified random sample of 15 of the 43 (seed 53, spanning all 8 families) was individually inspected against raw evidence fields. All 15 contained unambiguous external network indicators – routable external IPv4 addresses in `Net.WebClient` download cradles or `curl`/`Invoke-WebRequest` beaconing commands (12 of 15), or public staging/DDNS domains (`duckdns.org`, `sharepoint-files.net`, `example-cdn.net`; 3 of 15) – with zero cases of the filename, timestamp, or username false-positive patterns found and fixed during development (Section V). This is reported as a scale-consistency spot-check of the filter's behavior, not as a re-derivation of the Factor 1/2/3 percentages above, which remain as measured at $n = 80$.

B. Paired Clean-Versus-Contaminated Evaluation ($n = 320$)

To isolate the effect of log contamination from the base rate of structural feasibility, we conducted a paired evaluation on 320 alerts and their clean (uncontaminated) twins across two independent NCE runs (Run A and Run B). Table III details the post-filter no-actionable counts per injection family, showing near-identical distributions. The overall no-actionable counts with Wilson 95% intervals confirm this: pre-filter, Clean A scored 250/320 [0.7328, 0.8231] and Clean B 254/320 [0.7461, 0.8345], compared to Contaminated A 262/320 [0.7728, 0.8571] and Contaminated B 256/320 [0.7527, 0.8402]. Post-filter, the counts converged further: Clean A 275/320 [0.8170, 0.8932], Clean B 276/320 [0.8204, 0.8960], Contaminated A 277/320 [0.8239, 0.8987], and Contaminated B 273/320 [0.8102, 0.8877]. Because the metric counts "no actionable hypothesis" as success, a suppression payload would succeed by producing this outcome, so the metric must be read by attacker goal per family. However, the evaluation graph holds none of the GUIDE hosts or accounts (0/704 and 0/706 hypotheses name a host in the default graph; 0 of 1,503 non-T1071 hypotheses were FEASIBLE), so non-T1071 verdicts do not reflect SSE judgement.

A pairwise analysis of verdict changes reveals that the pipeline's outcome under contamination is showing

TABLE III
NO-ACTIONABLE-HYPOTHESIS COUNT BY INJECTION FAMILY ($n = 40$ EACH)

Injection Family	Clean A	Clean B	Contam A	Contam B
`authority_escalation`	38	38	38	38
`cross_field_split`	33	34	35	33
`direct_override`	35	35	35	35
`fabricated_evidence`	33	34	34	33
`fake_output_injection`	34	33	34	32
`native_format_mimicry`	36	36	35	36
`obfuscated_trigger`	33	33	33	32
`zero_imperative_evidence`	33	33	33	32

TABLE IV
APPROVALCLAIMDETECTOR FALSE-POSITIVE RATE, BEFORE AND AFTER STRUCTURAL FIX

Corpus	Overall	Dual-Signal	General
$n=30$ (keyword, pre-fix)	6.7% (2/30)	13.3% (2/15)	0.0% (0/15)
$n=30$ (structural fix)	0.0% (0/30)	0.0% (0/15)	0.0% (0/15)
$n=100$ (structural, scaled)	**1.0% (1/100)**	**2.0% (1/50)**	0.0% (0/50)

no detectable difference from its outcome on clean data. The noise floor between two runs of the same condition shows 3/320 verdict changes for Clean A vs Clean B (exact McNemar $b = 1, c = 2, p = 1.0000$) and 6/320 for Contaminated A vs B ($b = 5, c = 1, p = 0.2188$). Across conditions, verdict changes remain at this floor: 6/320 for Clean A vs Contam A ($b = 2, c = 4, p = 0.6875$), 2/320 for Clean A vs Contam B ($b = 2, c = 0, p = 0.5000$), 5/320 for Clean B vs Contam A ($b = 2, c = 3, p = 1.0000$), and 3/320 for Clean B vs Contam B ($b = 3, c = 0, p = 0.2500$). Because a non-significant McNemar test does not prove equality, we state this as no detectable effect, bounded within a few percentage points (at most about 4% of alerts changed verdict between conditions, the upper 95% Wilson bound for 6/320). This stability contrasts sharply with the LLM's own hypothesis generation, where technique sets differ in 53.1% of alerts (170/320) between the two clean runs and 43.4% (139/320) between the two contaminated runs. An exploratory T1071 confidence comparison (Clean A vs B: 25 lower / 16 higher, sign test $p = 0.2110$; Contam A vs B: 14/14, $p = 1.0000$; Clean vs Contam: 25/4 $p = 0.0001$, 31/13 $p = 0.0096$, 26/8 $p = 0.0029$, 24/13 $p = 0.0989$) was inconclusive, as runs differ in harness, day, and process model, pairs share alerts, and six comparisons were made. In earlier undefended triage tests on held-out alerts ($n = 10$ per family), this model was hijacked on three of the eight families used here (`fabricated_evidence` 60%, `zero_imperative_evidence` 30%, `cross_field_split` 10%) and on none of the others; the paired result above therefore comes from a model that the payloads rarely fool, which limits what it can show about the structural layers.

C. Clean-Alert False-Positive Assessment ($n = 60$)

Across 60 clean (non-contaminated) alerts, the non-T1071 false-positive rate is **0.0% (0/60)**. This zero is largely a consequence of the evaluation corpus: 100% of non-T1071 hypotheses were marked INFEASIBLE by SSE because the synthetic GUIDE-derived accounts carry no standing access-granting edges for identity-based techniques. The result confirms that SSE's graph-based feasibility check does not manufacture false FEASIBLE verdicts when graph edges are absent; it does not independently validate that the defense would hold on accounts seeded with real access grants (the latter is addressed for T1078, T1021, T1550, T1484, T1562 in Section VI-E). The T1071-specific FEASIBLE rate – alerts where SSE, absent any attack, still finds a structurally feasible C2 hypothesis because the alert genuinely resembles beaconing – falls from 16/60 (26.7%) to **10/60 (16.7%)** after the evidentiary filter. Comparing this to the contaminated corpus (where 56-60/400, or 14.0-15.0%, of alerts produce a T1071 FEASIBLE hypothesis), we state plainly: the T1071 FEASIBLE rate is largely a base-rate property of how GUIDE alerts describe network activity, not a contamination-specific effect. Attackers are not meaningfully raising this rate above what clean traffic already exhibits. Ten unsupported T1071 hypotheses were dropped across the clean corpus and individually verified against raw evidence, including resolution of a previously-flagged calibration failure: an alert receiving a 0.85-confidence T1071 hypothesis from evidence containing zero network indicators (the highest confidence among all weak-evidence cases in the corpus) is now correctly excluded.

D. ApprovalClaimDetector: Structural Discrimination at Scale

Detection retention holds at 9/10 unique fabricated-evidence attacks scoring HIGH risk; the 10th is a known false negative: its own text contains a distancing keyword within the 80-character window, causing the structural suppression condition to fire incorrectly. This is a genuine missed detection, not a correct suppression. Scaling the benign corpus from $n = 30$ to $n = 100$ – motivated by the same small-sample-bias concern as Section V, since a true $\sim$2% event rate has a $\sim$74% chance of appearing as zero occurrences at $n = 15$ – surfaces one residual false positive the smaller sample missed: an EDR analyst-note field using single-level `key="value"` structure, the same syntactic shape as 2 of the 10 real attacks in the corpus. We document this as a genuine representational ambiguity rather than pursue a further fix, since narrowing the detector to nested-quote structure only would demote both real attacks below detection threshold – trading one error for two.

E. RSEM Tie-Blindness on Egress-Only Techniques

Feeding all 56 structurally-feasible hypotheses from the full evaluation corpus through RSEM reveals that all four active actions (MONITOR_ONLY, REVOKE_SESSION, RESTRICT_PRIVILEGES, and QUARANTINE_ACCESS) score exactly 0.0000 in every single case – a four-way tie, confirmed individually across all 56 cases. This is *not*

RSEM correctly identifying MONITOR_ONLY as the superior action: per Table I, T1071's feasibility depends *only* on EGRESS edges between zones, yet none of the four active response actions (Section IV-F) ever mutate EGRESS edges – they act exclusively on GRANTS, HAS_PRIOR_ACCESS, and MEMBER_OF. Containment is therefore architecturally identical (zero) for all four candidate actions on *any* account, seeded or unseeded – we confirmed this holds even for a hand-constructed T1071 hypothesis against a fully seeded identity with standing HAS_PRIOR_ACCESS and GRANTS edges, ruling out sparse synthetic-account connectivity as the cause.

With four candidates genuinely tied, RSEM's stable sort (Section IV-F) resolves the tie by *input order in the candidate-action list*, not by any risk-based criterion. In our evaluation harness, MONITOR_ONLY was declared first in the candidate list, so all 56 cases resolved to MONITOR_ONLY and neither guardrail rule fired (AUTO_APPROVED). We verified this is an ordering artifact rather than a property of the scoring function itself by re-running the identical cases with REVOKE_SESSION declared first instead: composite scores were unchanged, but every case now selected REVOKE_SESSION, triggering the guardrail layer's hard-floor rule and routing all cases to PENDING_ANALYST_REVIEW – the opposite operational outcome, from an identical underlying risk assessment.

We therefore reframe this finding. Rather than the system safely and silently declining to act on ungroundable network alerts, RSEM lacks a defined tie-breaking policy for egress-only techniques, and the guardrail layer's behavior for this entire technique family is consequently governed by an implementation detail external to any risk calculation. The architectural root cause we originally identified still holds exactly as stated – no current action primitive can affect the edge type T1071's feasibility is gated on, so containment is identically zero by construction – but the operational consequence is a genuine blind spot, not a demonstrated safety property. The tie is confined to T1071 in our evaluation: across all 460 evaluated alerts (400 contaminated, 60 clean), zero non-T1071 hypotheses ever reached RSEM, since 100% of non-T1071 hypotheses were marked INFEASIBLE by SSE – the synthetic GUIDE-derived accounts carry no standing access-granting edges for identity-based techniques. On hand-constructed scenarios using seeded accounts, non-T1071 techniques break these ties legitimately: REVOKE_SESSION and RESTRICT_PRIVILEGES produce genuine nonzero containment (0.50–1.00) that strictly outranks MONITOR_ONLY, exactly as intended. We treat a defined tie-breaking policy for zero-containment techniques – for example, deterministically routing a genuine composite-score tie to mandatory review rather than relying on candidate-list order – as a required fix, not a future enhancement, and we no longer present the original silent-AUTO_APPROVED outcome as a strength of the current architecture. Non-T1071 techniques were separately validated against seeded accounts carrying the specific standing edges their SSE preconditions require (tests/test_rsem_seeded_scenarios.py), confirming each of the six other techniques correctly breaks the containment tie via nonzero, technique-appropriate containment scores that strictly outrank MONITOR_ONLY – ruling out T1071's tie-blindness as a general RSEM defect rather than a technique-specific architectural property.

VII. Discussion and Limitations

The network-egress validation ceiling is architectural, not a defect. SSE answers "does a structural path exist for this claim" – for genuine C2 telemetry, the honest answer is yes, and no realistic refinement of graph-based feasibility checking alone resolves this, because the underlying evidence genuinely supports feasibility. We consider this an important negative result: purely structural, non-LLM validation has a real, quantifiable ceiling for network-layer techniques specifically, distinct from and smaller than the ceiling for identity/privilege-based techniques. SSE's zero-false-positive result across $n = 60$ for non-T1071 techniques is a property of the synthetic evaluation corpus (accounts with no standing access-granting edges); on live enterprise accounts with real access grants, SSE would correctly produce FEASIBLE verdicts for legitimate access, which would need to be distinguished by downstream policy logic rather than structural feasibility alone.

Per-family rates at $n = 10$ are not directionally trustworthy, confirmed a third time. Scaling the contaminated evaluation from $n = 10$ to $n = 50$ per family (Table II) reproduced this project's bidirectional small-sample-bias finding rather than merely re-confirming the aggregate rate. Two families rose sharply once evidentiary-filtered (cross_field_split, 60.0%→82.0%; fake_output_injection, 70.0%→82.0%) while direct_override – a perfect 100.0% at $n = 10$, both baseline and filtered – fell to 88.0%/90.0%, and zero_imperative_evidence's filtered rate fell from a perfect 100.0% to 86.0%. The aggregate rate nonetheless held stable and rose slightly ($64/80$=80.0% → $326/400$=81.5% baseline; $67/80$=83.75% → $344/400$=86.0% filtered), consistent with the earlier finding (Section V) that n=10-scale estimates are unreliable *per-family* even when the aggregate proves stable – a distinction a paper reporting only $n = 10$ per-family rates would not have been able to draw.

To test whether NCE's non-deterministic LLM output affects the headline result, we re-ran hypothesis generation on the full 320-alert scale-up slice a second time under identical settings (same alert IDs, same model, no temperature parameter set). 162 of 320 alerts (50.6%) produced a different hypothesis set on the second run, confirming Gemini's output is not deterministic at default settings. Despite this, only 6 of 320 alerts (1.9%) changed final defense verdict, yielding 340/400 (85.0%) versus 344/400 (86.0%) on the original run – a 1.0-point difference. This indicates the structural defense outcome is substantially more stable than the LLM output feeding it, which is the property the architecture is designed to provide.

The evidentiary filter narrows but does not eliminate this gap, and does so by adding a second, complementary check

(evidentiary grounding at generation time) rather than attempting to strengthen SSE's structural check itself, which we argue is the more defensible design: it keeps SSE's contamination-immunity property (Section IV-C) fully intact while addressing a distinct failure mode in a separate, independently-testable component.

The ApprovalClaimDetector's residual single-level-field ambiguity illustrates a general risk in structure-based discrimination: attackers and legitimate tools can converge on the same low-level syntactic shape for different reasons. We chose not to resolve this via additional pattern engineering against a 100-log corpus, judging the overfitting risk to outweigh eliminating one residual false positive.

Scope limitations. The defense pipeline is untested on reasoning backends other than `gemini-3.1-flash-lite`. The GUIDE dataset, while designed to resemble production SIEM/EDR telemetry, is not live enterprise data. The Action/Playbook Layer is validated only in dry-run simulation against a synthetic graph, not against live SOAR/EDR execution. The 43 new post-filter-surviving cases at $n = 400$ were spot-checked (15/43) rather than exhaustively reclassified (Section VI-A); we report this limitation explicitly rather than presenting the $n = 80$ Factor 1/2/3 percentages as re-validated at scale, which they were not. At $n = 400$, the field `undefended_hijack` tracking whether NCE was actually fooled is absent from the 320-alert scale-up corpus; we therefore cannot report a per-alert hijack count at full scale and soften all claims about LLM manipulation to refer to the defense outcome rather than the LLM's internal state. Additionally, evaluation-harness graph state isolation was a real finding during development (Section V); all results reported in this paper reflect the corrected, read-only, order-independent evaluation.

VIII. Conclusion and Future Work

We show that a layered pipeline of independent, non-LLM structural verification components can prevent contaminated log evidence from producing an actionable security decision in 85.0-86.0% (340-344/400) of tested cases across two independent runs of 8 distinct injection strategies at a 5×-scaled sample, independently of the underlying LLM reasoning engine – while being explicit about the 14.0-15.0% residual gap, its architectural cause, and the per-family volatility this scale-up itself exposed, rather than presenting an inflated or falsely-precise success rate. Future work includes a defined RSEM tie-breaking policy for egress-only techniques where containment is architecturally zero for multiple candidate actions (Section VI-E), rather than relying on undefined candidate-action list order; technique-specific guardrail heuristics for external-target techniques lacking internal graph representation; exhaustive Factor 1/2/3 reclassification of the full $n = 400$ corpus; evaluation against additional reasoning backends; and integration with live SOAR/EDR execution to validate the Action/Playbook Layer beyond dry-run simulation.

References

[1] J. Roy and S. K. Singh, "AgentSOC: A Multi-Layer Agentic AI Framework for Security Operations Automation," in *Proc. 5th IEEE Int. Conf. on AI in Cybersecurity (ICAIC)*, Houston, TX, USA, Feb. 2026.